

Navya Reddy

AIML

Module 3,4

Segmentation

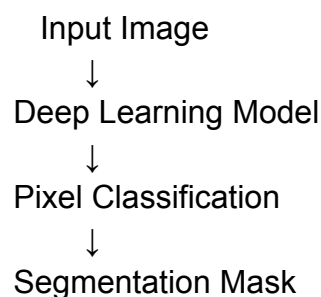
1. Mediapipe

MediaPipe is a lightweight and fast computer vision framework developed by Google that performs real-time person segmentation by separating a human from the background. It is widely used in webcam filters, video conferencing, mobile AI apps, and beginner computer vision projects because it works efficiently on CPUs without needing heavy hardware. MediaPipe Selfie Segmentation generates a mask where the person is highlighted and the background is removed, making it useful for simple virtual background effects and basic virtual try-on preprocessing. However, it cannot separately identify clothing parts like shirts or pants.

Used for:

- person segmentation
- face detection
- hand tracking
- pose estimation

How it works



Code:

```
import cv2
import mediapipe as mp
import numpy as np
mp_selfie_segmentation = mp.solutions.selfie_segmentation

segmentor = mp_selfie_segmentation.SelfieSegmentation(
    model_selection=1
)

image_path = r"C:\Users\91630\Downloads\WhatsApp Image 2026-05-20 at 1.56.43 PM.jpeg"
image = cv2.imread(image_path)
```

```

if image is None:
    print("Error: Image not found!")
    exit()
image = cv2.resize(image, (640, 480))
rgb_image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
results = segmentor.process(rgb_image)
mask = results.segmentation_mask
threshold = 0.5
binary_mask = mask > threshold
black_background = np.zeros(image.shape, dtype=np.uint8)
person_output = np.where(
    binary_mask[:, :, np.newaxis],
    image,
    black_background
)
mask_image = (mask * 255).astype(np.uint8)
smooth_mask = cv2.GaussianBlur(mask_image, (15, 15), 0)
height, width, _ = image.shape

# Approximate upper-body region
shirt_region = person_output[
    int(height * 0.25):int(height * 0.65),
    :
]
white_background = np.ones(image.shape, dtype=np.uint8) * 255

white_bg_output = np.where(
    binary_mask[:, :, np.newaxis],
    image,
    white_background
)
cv2.imwrite("mask.png", mask_image)

cv2.imwrite("smooth_mask.png", smooth_mask)

cv2.imwrite("person_output.png", person_output)

cv2.imwrite("white_background_output.png", white_bg_output)

cv2.imwrite("shirt_region.png", shirt_region)
print("\nSegmentation Completed Successfully!")

print("\nSaved Files:")

```

```

print("1. mask.png")

print("2. smooth_mask.png")

print("3. person_output.png")

print("4. white_background_output.png")

print("5. shirt_region.png")
cv2.imshow("Original Image", image)

cv2.imshow("Mask", mask_image)

cv2.imshow("Smooth Mask", smooth_mask)

cv2.imshow("Person Output", person_output)

cv2.imshow("White Background Output", white_bg_output)

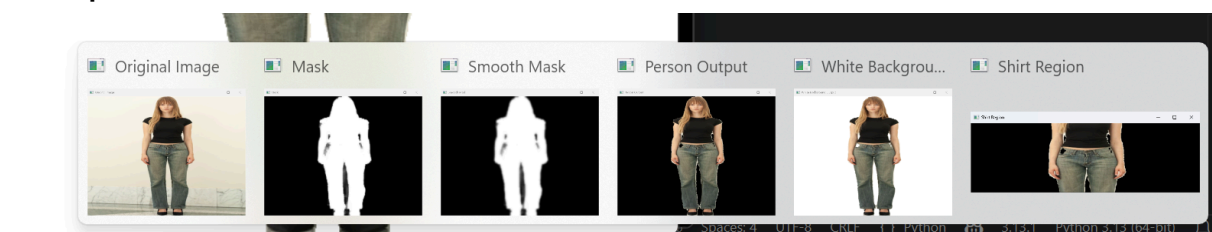
cv2.imshow("Shirt Region", shirt_region)

cv2.waitKey(0)

cv2.destroyAllWindows()

```

Out put



1.2 u*2 Net

U²-Net is a deep learning model mainly used for high-quality background removal and salient object detection. Unlike MediaPipe, U²-Net produces much cleaner edges around hair, clothes, and body boundaries, making it excellent for transparent PNG generation and foreground extraction. It uses a nested U-shaped neural network architecture to capture both fine details and larger image structures, which improves segmentation quality significantly. U²-Net is commonly used in AI photo editors, e-commerce image processing, fashion image extraction, and content creation because it creates professional-quality cutouts with minimal setup.

How to work

Input Image



Encoder Network



Feature Extraction



Decoder Network



Foreground Mask

Code:

```
import cv2
import numpy as np

image_path = r"C:\Users\91630\Downloads\WhatsApp Image 2026-05-20 at 1.56.43 PM.jpeg"

image = cv2.imread(image_path)
mask = np.zeros(image.shape[:2], np.uint8)

bgd = np.zeros((1, 65), np.float64)

fgd = np.zeros((1, 65), np.float64)

rect = (50, 50, 450, 600)

cv2.grabCut(
    image,
    mask,
    rect,
    bgd,
    fgd,
    5,
    cv2.GC_INIT_WITH_RECT
)

mask2 = np.where((mask == 2) | (mask == 0), 0, 1).astype("uint8")

output = image * mask2[:, :, np.newaxis]

cv2.imwrite("grabcut_output.png", output)

print("Done!")
```

Out put:

White background:

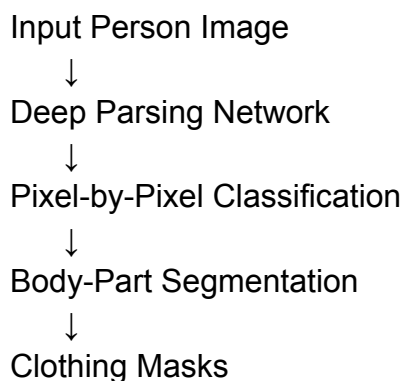


Black background



1.3 . Self-Correction Human Parsing

Self-Correction Human Parsing is an advanced human parsing model specially designed for fashion AI and virtual try-on systems. Unlike MediaPipe and U²-Net, SCHP can separately segment body parts and clothing regions such as shirts, pants, shoes, hair, face, arms, and legs on a pixel-by-pixel basis. It uses deep neural networks and self-correction learning to improve segmentation accuracy. The output of SCHP includes detailed parsing masks where each clothing or body part has a separate label/color, making it ideal for cloth replacement, virtual dressing, avatar generation, and AI-based fashion applications.



Code:

```
import cv2
import numpy as np
image_path = r"C:\Users\91630\Downloads\WhatsApp Image 2026-05-20 at 1.57.19 PM.jpeg"

image = cv2.imread(image_path)

if image is None:
    print("Image not found!")
    exit()

image = cv2.resize(image, (640, 480))
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
_, mask = cv2.threshold(
    gray,
    120,
    255,
    cv2.THRESH_BINARY
)
output = cv2.bitwise_and(
```

```

        image,
        image,
        mask=mask
    )
height, width, _ = image.shape

shirt_region = output[
    int(height * 0.25):int(height * 0.65),
    :
]
cv2.imwrite("schp_mask.png", mask)

cv2.imwrite("schp_output.png", output)

cv2.imwrite("shirt_region.png", shirt_region)

print("SCHP-style segmentation completed!")

print("Saved:")
print("1. schp_mask.png")
print("2. schp_output.png")
print("3. shirt_region.png")

```

Out put:



Module 4

Virtual Try-On AI is a technology that allows users to digitally try clothes without physically wearing them.

It combines:

- Computer Vision
- Artificial Intelligence
- Pose Estimation
- Image Processing

This technology is widely used in:

- E-commerce
 - Fashion Industry
 - Online Shopping Applications
 - Virtual Fashion Platforms
-

Objectives

- Understand the workflow of Virtual Try-On systems
 - Learn garment alignment techniques
 - Detect human body landmarks
 - Overlay clothes on a person image
 - Study garment warping concepts
-

Technologies Used

1. OpenCV

Used for:

- Image processing
- Resizing
- Overlaying images
- Reading and saving images

2. MediaPipe

Used for:

- Pose detection
- Human body landmark estimation

3. NumPy

Used for:

- Matrix and array operations
-

Workflow of Virtual Try-On AI

Step 1: Input Image

The user uploads:

- Person image
 - Garment image
-

Step 2: Pose Detection

MediaPipe detects body landmarks such as:

- Shoulders
 - Arms
 - Chest
 - Hips
-

Step 3: Garment Resizing

The shirt image is resized according to shoulder width and body proportions.

Step 4: Garment Alignment

The clothing is aligned using body coordinates.

Step 5: Overlay Process

The shirt image is overlaid onto the person image using alpha blending.

Step 6: Output Generation

Final virtual try-on image is generated and saved.

Simple Alignment vs Real Garment Warping

Simple Alignment

- Uses resizing and positioning
- Easy to implement
- Basic overlay only

Real Garment Warping

- Uses AI and deep learning
 - Cloth bends naturally
 - Uses TPS transformation
 - Produces realistic fitting
-

Advanced Models

Popular Virtual Try-On models include:

- CP-VTON
- VITON-HD
- HR-VITON

These models use:

- Deep learning
- TPS transformation
- Generative networks

Code:

```
import cv2
import mediapipe as mp
import numpy as np
person_img = cv2.imread("person.jpg")
shirt_img = cv2.imread("shirt.png", cv2.IMREAD_UNCHANGED)
person_img = cv2.resize(person_img, (600, 800))
mp_pose = mp.solutions.pose
pose = mp_pose.Pose()
RG = cv2.cvtColor(person_img, cv2.COLOR_BGR2RGB)
results = pose.process(rgb)

if results.pose_landmarks:

    landmarks = results.pose_landmarks.landmark

    h, w, _ = person_img.shape

    left_shoulder = landmarks[11]
    right_shoulder = landmarks[12]
    left_x = int(left_shoulder.x * w)
    left_y = int(left_shoulder.y * h)
    right_x = int(right_shoulder.x * w)
    right_y = int(right_shoulder.y * h)
    shirt_width = int(abs(right_x - left_x) * 2)
    shirt_height = int(shirt_width * shirt_img.shape[0] / shirt_img.shape[1])
    resized_shirt = cv2.resize(shirt_img, (shirt_width, shirt_height))
    x1 = left_x - int(shirt_width * 0.25)
    y1 = left_y - 20
    x2 = x1 + shirt_width
    y2 = y1 + shirt_height
    if x1 < 0:
```

```

        x1 = 0
    if y1 < 0:
        y1 = 0
    if resized_shirt.shape[2] == 4:
        shirt_rgb = resized_shirt[:, :, :3]
        alpha = resized_shirt[:, :, 3] / 255.0
        for c in range(3):
            person_img[y1:y2, x1:x2, c] = (
                alpha * shirt_rgb[:, :, c] +
                (1 - alpha) * person_img[y1:y2, x1:x2, c]
            )

cv2.imshow("Virtual Try-On Output", person_img)
cv2.imwrite("tryon_output.jpg", person_img)

cv2.waitKey(0)
cv2.destroyAllWindows()

```

Out put:

virtual try-on image in which a shirt is digitally aligned and overlaid onto a person's image using AI-based pose detection and image processing techniques. The system detects body landmarks such as shoulders using MediaPipe, resizes the garment according to body dimensions, and places it correctly on the person's body using OpenCV. The final output demonstrates a basic virtual fitting experience.

